

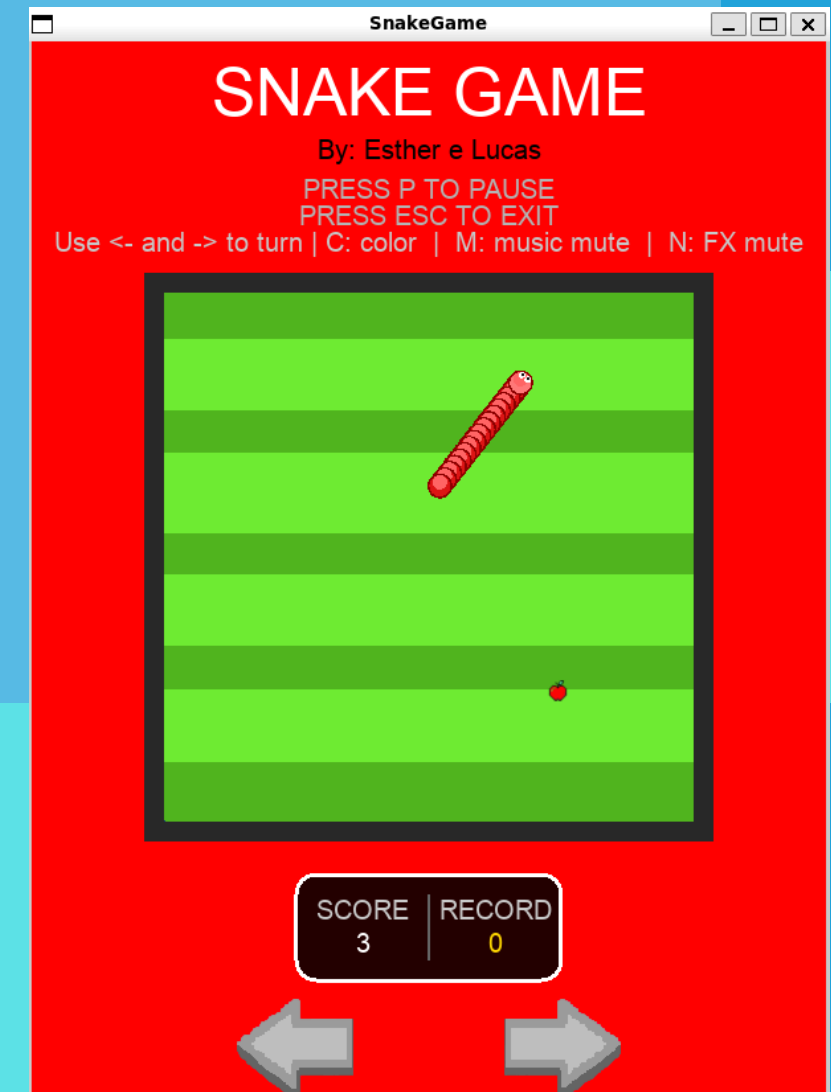


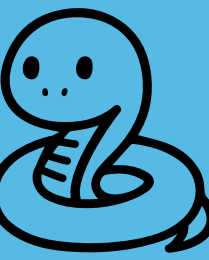
SNAKE GAME

By Esther e Lucas



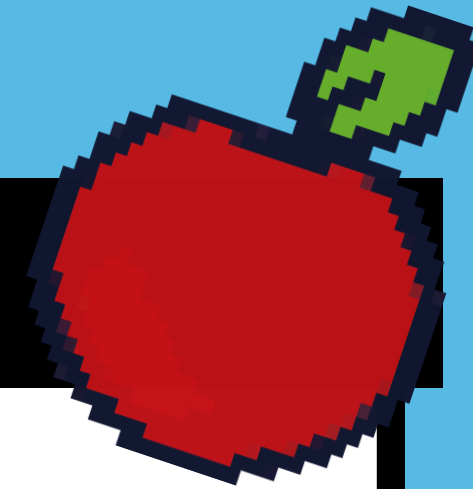
A drawing of a mobile phone with a green screen showing a simple line drawing of a person. The phone is grey with a black screen. The screen displays a green line drawing of a person standing with arms outstretched. Above the screen is a small speaker grille. Below the screen is a navigation pad. The phone has a standard numeric keypad with letters on the buttons. The background is a solid blue color.





OBJETIVO

Procuramos desevelopver:



Projeto funcional em multiplataformas - Docker

Código documentado padronizado - Doxygen

Interface gráfica - Allegro 5

Programa Orientado a Objetos - C++17

ESTRUTURA DE DIRETÓRIOS



include/SnakeGame/

|—— Game.hpp

|—— Entity.hpp

|—— Vector2D.hpp

|—— Types.hpp

|—— Apple.hpp

|—— Field.hpp

|—— Snake.hpp

|—— ConsoleHUD.hpp

|—— Sound.hpp

Headers públicos

Classe principal

Base para entidades

Matemática 2D

Configurações e tipos

Maçã coletável

Campo de jogo

Cobra jogável

Interface

Sistema de áudio

ESTRUTURA DE DIRETÓRIOS



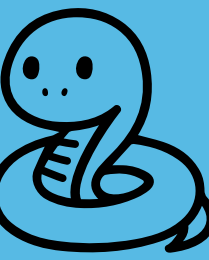
src/

- |—— Game.cpp
- |—— Snake.cpp
- |—— Apple.cpp
- |—— Field.cpp
- |—— ConsoleHUD.cpp
- |—— Sound.cpp
- |—— main.cpp
- |—— main_text_linux.cpp
- |—— main_no_audio.cpp
- |—— main_text.cpp

- # Implementações
- # Loop principal
- # Movimento e colisão
- # Spawning de comida
- # Renderização do campo
- # HUD e placar
- # Efeitos e música
- # main principal
- # Ponto de entrada linux versão texto
- # Ponto de entrada para versão executável sem áudio
- # Versão terminal windows

src/private

- |—— ResourceManager.hpp
- |—— AllegroLoaders.hpp
- |—— AllegroGfx.hpp



CLASSES

Foram usadas 8 classes

Game: classe principal

ConsoleHUD: interface do usuário

Field: campo de jogo

SoundBank: sistema de áudio

Apple: maçã coletável

AllegroInit: inicialização do Allegro

Snake: cobra controlável

Entity: classe base abstrata



GAME



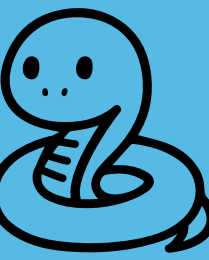
Exemplo

```
// Game.cpp -  
  
void Game::run() {  
    while (running_) {  
        ALLEGRO_EVENT ev{};  
        al_wait_for_event(queue_, &ev);  
        handleEvent(ev);    // Processa input  
  
        if (ev.type == ALLEGRO_EVENT_TIMER) {  
            updateLogic();    // Atualiza jogo  
            render();        // Renderiza frame  
        }  
    }  
}
```

Executa o loop principal do jogo, processando eventos, atualizando a lógica e renderizando cada frame.

Explicação

Classe principal que coordena todo o jogo, controlando o loop principal, os estados (menu, jogando, game over) e orquestrando todas as outras entidades



SNAKE

Classe que controla a cobra do jogador, gerenciando seu movimento, crescimento por comer maçãs e detecção de colisões consigo mesma

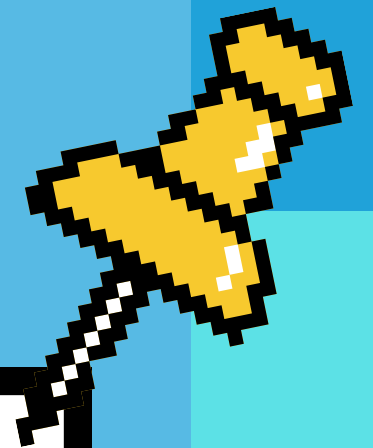
EXEMPLO:

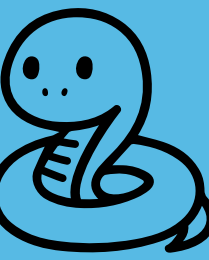
```
// Snake.cpp - Movimento REAL da cobra
void Snake::update(double /*dt*/) {
    const double rad = degToRad(headAngleDeg_);
    const double vx = std::cos(rad) * speedModule_;
    const double vy = std::sin(rad) * speedModule_;

    // Move a cobra: cada segmento segue o anterior
    for (int i = storedSize_-1; i > 0; --i)
        body_[i] = body_[i-1];

    // Atualiza posição da cabeça
    body_[0].pos.x = body_[1].pos.x + vx;
    body_[0].pos.y = body_[1].pos.y + vy;
}
```

Atualiza a posição da cobra frame a frame, fazendo cada segmento seguir o anterior e movendo a cabeça baseada no ângulo atual





CONSOLE HUD - INTERFACE

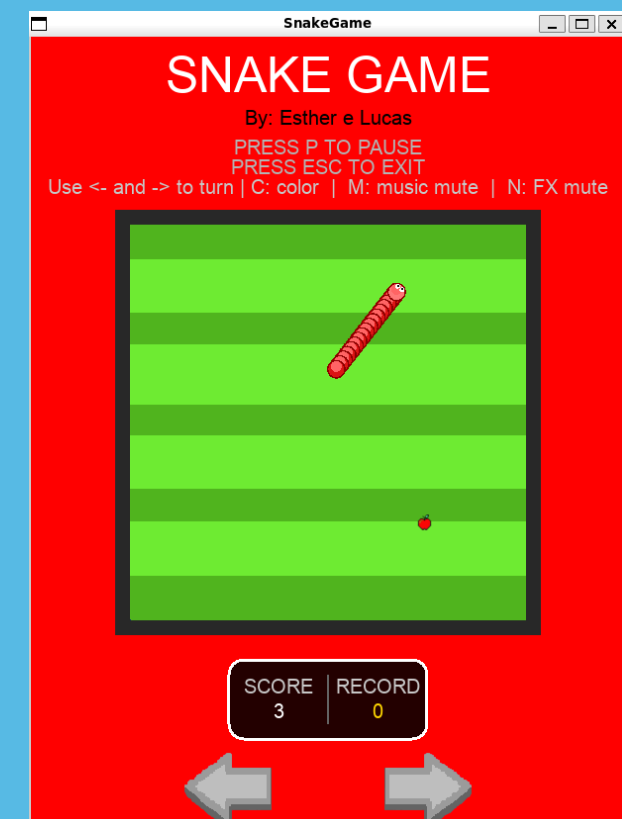
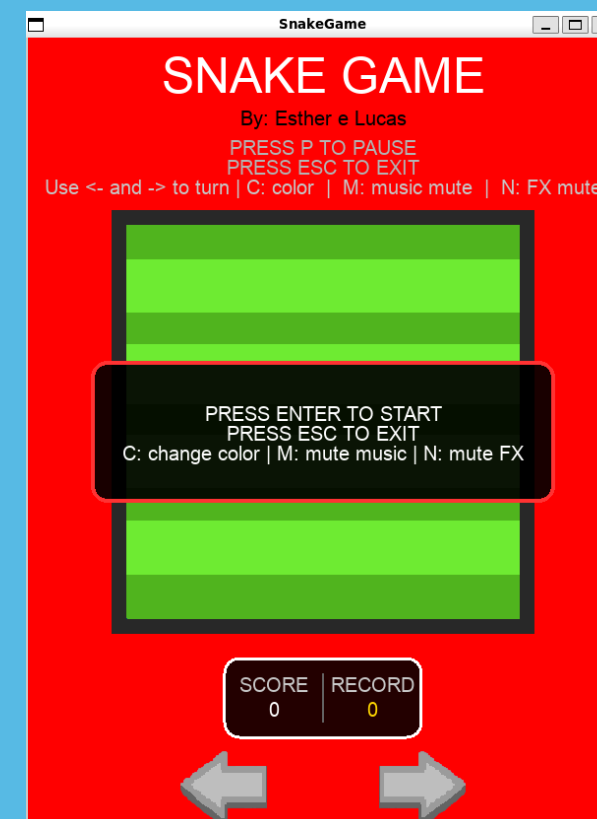
Exemplo

```
// Game.cpp -  
void Game::updateLogic() {  
    if (state_ == GameState::Playing && !paused_)  
    {  
        // Atualiza HUD com scores em tempo real  
        hud_->setScore(snake_->score()/10);  
        hud_->setHighScore(highScore_);  
    }  
}
```

Atualiza a pontuação no HUD enquanto o jogo está rodando, mostrando instantaneamente quando a cobra come maçãs

Explicação

Classe responsável por mostrar todas as informações do jogo na tela, como pontuação, recorde e controles visuais, criando uma interface de usuário clara



SOUNDBANK

Classe que gerencia todos os sons do jogo, controlando música de fundo e efeitos sonoros como "comer maçã" e "game over"

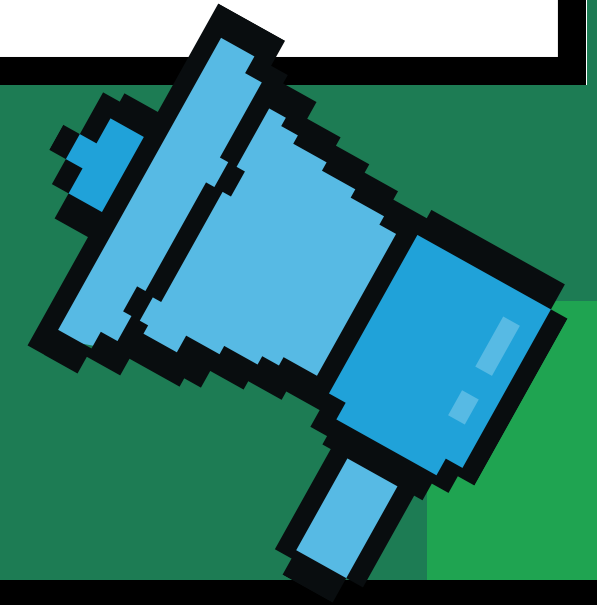
EXEMPLO DE POLIMORFISMO NO SOUDBANK:

```
void SoundBank::playEat(float vol) {  
    if (!ok_ || fxMuted_ || !eat_) return;  
    al_play_sample(eat_, vol, 0.0f, 1.0f, ALLEGRO_PLAYMODE_ONCE, nullptr);  
}  
  
void SoundBank::playGameOver(float vol) {  
    if (!ok_ || fxMuted_ || !gameOver_) return;  
    al_play_sample(gameOver_, vol, 0.0f, 1.0f, ALLEGRO_PLAYMODE_ONCE, nullptr);  
}  
// MESMA ação "play", SOM diferente!
```

EXEMPLO DE SOUDBANK:

```
// SoundBank.cpp  
void SoundBank::playEat(float vol) {  
    if (!ok_ || fxMuted_ || !eat_) return;  
    al_play_sample(eat_, vol, 0.0f, 1.0f,  
        ALLEGRO_PLAYMODE_ONCE, nullptr);  
}
```

Toca o som de "comer maçã" quando a cobra colide com uma maçã, mas só se os efeitos sonoros não estiverem mutados





STRUCTS

Foram usados 8 structs



Vector2D: coordenadas espaciais

Segment: corpo articulado da cobra

AllegroGfx: inicialização gráfica

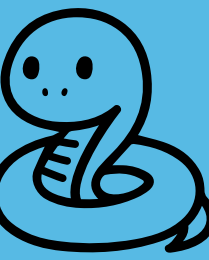
BitmapLoader: carregador de imagens

FrontSpecLoader: carregador de fontes

BitmapUnloader: liberador de memória

FontUnloader: liberador de fontes

SnakePalette: paleta de cores



VECTOR 2D

Exemplo

```
// Vector2D - define onde as coisas estão  
// Snake.cpp - Cálculo de colisão com maçã  
Vector2D h = snake_->headPosition(); //  
Posição da cabeça  
Vector2D a = apple_->position(); // Posição da  
maçã  
const double dx = h.x - a.x, dy = h.y - a.y;  
  
// Apple.cpp - Spawn aleatório usando  
Vector2D  
void Apple::spawnRandom() {  
    pos_.x = dx(rng); // Coordenada X aleatória  
    pos_.y = dy(rng); // Coordenada Y aleatória  
}
```

Explicação

É usado por todas as entidades do jogo para representar posições, movimentos e cálculos espaciais. Sem ele, nada teria posição no jogo



SEGMENT

É a unidade fundamental que compõe a cobra, definindo sua estrutura e movimento característico.

EXEMPLO:

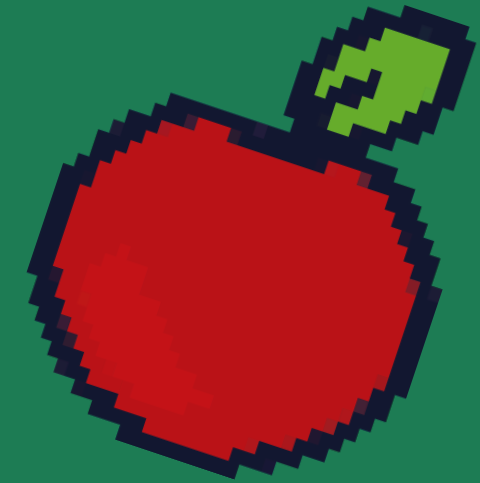
// Segment - **define COMO a cobra é construído**

// Snake.cpp - Estrutura interna da cobra

std::vector<Segment> body_; // Array de segmentos

// Cada Segment tem:

```
struct Segment {  
    Vector2D pos{};    // POSIÇÃO do segmento  
    double size{8.0}; // TAMANHO do segmento  
    double angleDeg{0.0}; // ÂNGULO do segmento  
};
```

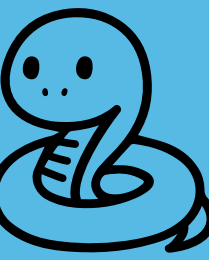


EXEMPLO DE SOUDBANK:

```
// SoundBank.cpp  
void SoundBank::playEat(float vol) {  
    if (!ok_ || fxMuted_ || !eat_) return;  
    al_play_sample(eat_, vol, 0.0f, 1.0f,  
        ALLEGRO_PLAYMODE_ONCE, nullptr);  
}
```

Toca o som de "comer maçã" quando a cobra colide com uma maçã, mas só se os efeitos sonoros não estiverem mutados

ARQUITETURA POR CAMADAS



Model

- Game
- Field
- Apple
- Snake
- Entity

View

- ConsoleHUD
- Render System
- SoundBank

Controller

- InputHandler
- Game Controller
- Collision Manager

Utilitários

- Vector2D
- Loaders
- Enums

Infraestrutura

- Templates
- Policies
- Resource Manager
- AllegroInit

Modelo MVC:

INPUT → CONTROLLER → MODEL → VIEW → OUTPUT
(Teclas) → (Processa) → (Atualiza Estado) → (Renderiza)



CONCEITOS DE POO

Abstração

Usado para
mostrar uma
interface
simples ao
usuário

Encapsulamento

Usado para
proteger os
dados
internos de
uma classe

Herança

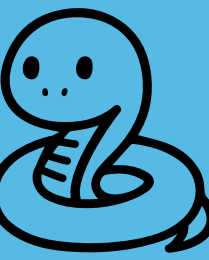
Uma classe
herda as
características
de outra

Polimorfismo

Mesmo método
se comportar
de forma
diferente em
classes
distintas

Templates

Usado para
criar classes
e funções
genéricas



ABSTRAÇÃO

Exemplo

```
// Vector2D.hpp - ABSTRAI matemática complexa
struct Vector2D {
    double x, y;

    // INTERFACE SIMPLES para operações complexas
    Vector2D operator+(const Vector2D& o) const {
        return Vector2D(x + o.x, y + o.y);
    }

    double norm() const { // Distância - matemática complexa escondida
        return std::sqrt(x*x + y*y);
    }
};

// USO SIMPLES:
Vector2D posicaoCobra = {100, 200};
Vector2D movimento = {5, 0};
Vector2D novaPosicao = posicaoCobra + movimento; // mostrado na prática
double distancia = posicaoCobra.norm();        // mostrado na prática
```

Ponto positivo

Nos permite mudar completamente a implementação da cobra sem afetar o resto do jogo, mantendo o código limpo e seguro

ENCAPSULAMENTO



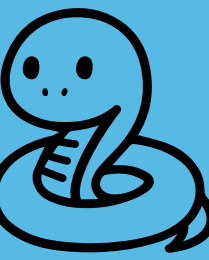
Exemplo

```
// Snake.hpp  
  
class Snake {  
  
public:  
  
    // INTERFACE PÚBLICA - o que outros podem usar  
  
    void turnLeft();  
  
    void turnRight();  
  
    void grow();  
  
    Vector2D headPosition() const;  
  
private:  
  
    // DETALHES ESCONDIDOS - ninguém de fora mexe  
  
    std::vector<Segment> body_;  
  
    double headAngleDeg_;  
  
    double rotationSpeed_;
```

Ponto positivo

Proteção: apenas o necessário fica visível, evitando alterações no código

HERANÇA



Exemplo

// Entity.hpp - CLASSE PAI

```
class Entity {  
public:  
    virtual ~Entity() = default;  
    virtual void update(double dt) {} // Pode ser sobrescrito  
    virtual void draw() const = 0; // DEVE ser implementado  
};
```

// Field.hpp - CLASSE FILHA

```
class Field : public Entity { // Field É UM Entity  
public:  
    void draw() const override; // IMPLEMENTA o método  
obrigatório
```

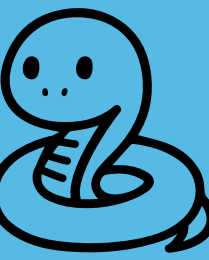
Classes filhas

- Field, Apple e Snake são filhas de Entity

Vantagem

```
// No Game.cpp - posso tratar todos como Entity  
void Game::render() {  
    field_->draw(); // Chama Field::draw()  
    apple_->draw(); // Chama Apple::draw()  
    snake_->draw(); // Chama Snake::draw()  
    // Todos são Entity, então todos têm draw()!  
}
```

Permite tratar todas as entidades do jogo de forma uniforme no game loop, usando `entity->draw()` sem precisar saber se é uma cobra, maçã ou campo.



POLIMORFISMO

Exemplo

```
//void Game::render() {  
  // MESMA LINHA DE CÓDIGO, COMPORTAMENTOS DIFERENTES!  
  field_->draw(); // Vira Field::draw() - desenha QUADRADO  
  apple_->draw(); // Vira Apple::draw() - desenha CÍRCULO  
  snake_->draw(); // Vira Snake::draw() - desenha SEGMENTOS  
  // Todos são Entity, mas cada um desenha DO SEU JEITO!
```

o mesmo draw() se transforma em comportamentos diferentes para cada objeto

Ponto positivo

Permite que o game loop renderize todas as entidades com uma única linha de código, mesmo que cada uma tenha sua própria implementação específica de como se desenhar na tela



TEMPLATES

Exemplo com Template

```
// ResourceManager.hpp - FUNCIONA COM QUALQUER TIPO
template<typename Key, typename Resource, typename Loader, typename Unloader>
class ResourceManager {
public:
    Resource get(const Key& key) {
        auto it = cache_.find(key);
        if (it != cache_.end()) return it->second; // Já tem? Reusa!

        Loader loader;
        Resource res = loader(key); // Carrega do disco
        cache_.emplace(key, res); // Guarda no cache
        return res;
    }
private:
    std::unordered_map<Key, Resource> cache_;
};

// ESPECIALIZAMOS PARA DIFERENTES TIPOS:
using BitmapRM = ResourceManager<std::string, ALLEGRO_BITMAP*,
                                BitmapLoader, BitmapUnloader>;
using FontRM = ResourceManager<std::string, ALLEGRO_FONT*,
                               FontSpecLoader, FontUnloader>;
```

Ponto positivo

Elimina código duplicado permitindo que um único gerenciador de recursos sirva para diferentes tipos, mantendo a type safety do compilador e melhorando a reutilização.



TEMPLATES

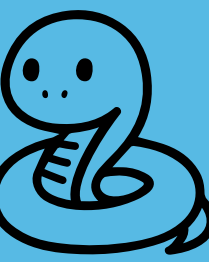
Exemplo sem Template

```
// GERENCIADORES SEPARADOS
class BitmapManager {
    ALLEGRO_BITMAP* get(const string& key) {
        if (cache_.count(key)) return cache_[key];
        return cache_[key] = al_load_bitmap(key.c_str());
    }
    unordered_map<string, ALLEGRO_BITMAP*> cache_;
};

class FontManager {
    ALLEGRO_FONT* get(const string& key) {
        if (cache_.count(key)) return cache_[key];
        return cache_[key] = al_load_ttf_font(key.c_str(), 16, 0);
    }
    unordered_map<string, ALLEGRO_FONT*> cache_;
};
```

Ponto negativo

Duplicação de código
força manutenção
múltipla e viola o princípio
DRY, criando classes
idênticas para cada tipo.
Com templates, uma
única classe genérica
serve para bitmaps,
fontes, sons e qualquer
outro recurso



ALLEGRO 5 - BASE GRÁFICA

Exemplo

```
// Inicialização simples de display e bitmap  
ALLEGRO_DISPLAY* display = al_create_display(800, 600);  
ALLEGRO_BITMAP* snake_head =  
al_load_bitmap("assets/snake_head.png");  
al_draw_bitmap(snake_head, 100, 100, 0);  
al_flip_display();
```

Escolha

Biblioteca de código aberto especializada em criar jogos 2D, fornecendo tudo que precisamos para gráficos, áudio e controles

Ponto positivo

Fornece uma API unificada e de alto nível para gráficos 2D, áudio e input



THANK
YOU!

